UNIVERSIDADE DO ESTADO DO RIO DE JANEIRO
INSTITUTO POLITÉCNICO

Modelagem Computacional (IPRJ01-10760/IPRJ01-11873)
Professor: Gustavo Barbosa Libotte

Ana Clara da Silva Vaz
Danillo Silva Linhares

OTIMIZAÇÃO DO CUSTO DE UM TANQUE CILÍNDRICO PELO MÉTODO DO GRADIENTE COM FUNÇÃO DE PENALIZAÇÃO

Relatório do Trabalho Computacional

Nova Friburgo
2026

SUMÁRIO

1 INTRODUÇÃO

O dimensionamento de reservatórios de armazenamento é uma tarefa recorrente em projetos de engenharia, pois a geometria escolhida determina diretamente a quantidade de material empregada e, portanto, o custo de construção. Para um tanque cilíndrico, qualquer volume pode ser obtido com infinitas combinações de raio e altura, e cada combinação conduz a áreas de base, parede lateral e teto distintas. Escolher a combinação de menor custo que ainda atenda ao volume exigido é um problema típico de otimização com restrição.

Neste trabalho, o problema é resolvido por uma combinação de três técnicas numéricas. A primeira é a função de penalização exterior, que transforma o problema com restrição de desigualdade em um problema irrestrito ao somar ao custo uma multa quadrática sempre que o volume mínimo é violado. A segunda é a aproximação das derivadas parciais por diferenças finitas adiantadas, que dispensa o cálculo analítico do gradiente. A terceira é o método do gradiente, que percorre iterativamente a direção de máxima descida da função auxiliar até que o gradiente se anule dentro de uma tolerância.

O objetivo é encontrar o raio e a altura que minimizam o custo de um tanque com volume mínimo de 500 m³, avaliar a convergência do algoritmo e discutir a influência da constante de penalidade μ e do tamanho do passo α sobre a solução e sobre o esforço computacional. O texto está organizado em Problema, Metodologia, Resultados e Conclusão, e o código-fonte completo, escrito em Python, está reproduzido no Apêndice A.

2 PROBLEMA

2.1 Função objetivo

O tanque é composto por base circular, parede lateral e teto circular, cujos custos de fabricação são, respectivamente, R$ 150,00/m², R$ 100,00/m² e R$ 200,00/m². Sendo r o raio e h a altura, ambos em metros, o custo total a ser minimizado é

$$f(r, h) = 150(\pi r^2) + 100(2\pi rh) + 200(\pi r^2) = 350\pi r^2 + 200\pi rh \quad (1)$$

2.2 Restrição de volume

Sem restrição, o mínimo de f ocorre em $r = h = 0$, que corresponde a um tanque que não armazena nada. Por isso, exige-se um volume mínimo $V_{\min} = 500$ m³. Como o volume do cilindro é $V = \pi r^2 h$, a restrição de desigualdade na forma padrão é

$$g(r, h) = 500 - \pi r^2 h \leq 0 \quad (2)$$

Assim, o problema consiste em minimizar $f(r, h)$ sujeito a $g(r, h) \leq 0$, com $r > 0$ e $h > 0$.

2.3 Solução analítica de referência

O problema admite solução analítica simples, que será usada apenas para validar o resultado numérico. Como f cresce com h para qualquer r positivo, não há vantagem em construir um tanque maior que o necessário, e a restrição está ativa no ótimo, isto é, $\pi r^2 h = 500$. Isolando h e substituindo em f, obtém-se uma função de uma única variável,

$$f(r) = 350\pi r^2 + 100000 / r \quad (3)$$

cuja derivada se anula quando $r^3 = 100000/(700\pi)$. Resulta $r^* \approx 3{,}5693$ m, $h^* \approx 12{,}4926$ m e custo mínimo $f^* \approx$ R$ 42.024,95. Como $f(r)$ é convexa para $r > 0$, esse ponto é o mínimo global.

3 METODOLOGIA

3.1 Função de penalização

Para tratar a restrição, minimiza-se a função auxiliar

$$P(r, h) = f(r, h) + \mu \cdot [\max(0, g(r, h))]^2 \quad (4)$$

Quando o volume é maior ou igual a 500 m³, g é não positivo, o termo de penalidade se anula e $P = f$. Quando o volume é insuficiente, g é positivo e a penalidade quadrática eleva o valor de P, o que empurra o algoritmo de volta à região viável. No código, essa função é implementada em `P(r, h, mu)`, que chama `f` e `g`.

3.2 Derivadas por diferenças finitas

O gradiente de P é calculado numericamente por diferenças finitas adiantadas, com passo $\Delta = 10^{-5}$:

$$\partial P/\partial r \approx [P(r + \Delta, h) - P(r, h)] / \Delta \quad (5)$$

$$\partial P/\partial h \approx [P(r, h + \Delta) - P(r, h)] / \Delta \quad (6)$$

A função `gradiente` avalia P uma vez no ponto atual e uma vez em cada ponto perturbado, ou seja, três avaliações de P por iteração. O erro dessa aproximação é proporcional a Δ, de modo que o valor adotado equilibra o erro de truncamento e o erro de arredondamento da aritmética de ponto flutuante.

3.3 Laço iterativo do método do gradiente

A partir de um chute inicial (r_0, h_0), as variáveis são atualizadas em sentido contrário ao gradiente:

$$r^{k+1} = r^{k} - \alpha\, \partial P/\partial r, \quad h^{k+1} = h^{k} - \alpha\, \partial P/\partial h \tag{7}$$

com μ e α constantes ao longo do processo. A função `metodo_gradiente` repete os seguintes passos:

a) calcula o gradiente numérico de P no ponto atual;

b) encerra com sucesso se a norma do gradiente for menor que a tolerância de 1,0;

c) atualiza r e h pelas equações de atualização;

d) encerra sem convergência se o número de iterações atingir 3.000.000 ou se r ou h deixarem de ser finitos ou ultrapassarem 10^6 em módulo (divergência).

A cada 100 iterações o valor de P é armazenado, para que a evolução da convergência possa ser traçada. A função `resumo` executa o método e devolve r, h, o número de iterações, o volume V, o custo f no ponto final e o indicador de convergência.

3.4 Experimentos realizados

Foram realizados três experimentos, todos em Python com as bibliotecas *math*, *random*, *NumPy* e *Matplotlib*. No primeiro, o método é executado 10 vezes com $\mu = 100$ e $\alpha = 8 \times 10^{-8}$, com chutes iniciais sorteados uniformemente, com r entre 0,1 e 10 m e h entre 0,1 e 20 m. Uma execução é considerada viável se o método convergiu e se a violação do volume, $500 - V$, não excede 0,5 m³. Para que os resultados sejam reprodutíveis, a semente do gerador aleatório foi fixada em 42 (linha `rd.seed(42)` no início da função `main`). O código completo encontra-se no Apêndice A.

No segundo experimento, parte-se do chute fixo (5, 10) e varia-se μ entre 10, 30, 100 e 300, com $\alpha = 8 \times 10^{-6}/\mu$. A escolha de α inversamente proporcional a μ decorre do fato de que a curvatura da penalidade cresce linearmente com μ, o que reduz proporcionalmente o maior passo estável. No terceiro experimento, mantém-se $\mu = 100$ e varia-se α entre 3×10^{-8}, 8×10^{-8}, $1{,}1 \times 10^{-7}$ e 3×10^{-7}, novamente a partir de (5, 10).

4 RESULTADOS

4.1 Ponto ótimo obtido

A Tabela 1 reúne as 10 execuções com chutes iniciais aleatórios. Todas convergiram para uma solução viável. Nove das dez execuções terminaram em $r \approx 3{,}5595$ m e $h \approx 12{,}5545$ m, com volume de 499,719 m³ e custo de R$ 42.009,47. A execução 3 parou ligeiramente antes, em $r = 3{,}5573$ m e $h = 12{,}5698$ m, com custo de R$ 42.009,62. O custo médio das execuções foi de R$ 42.009,48, e o número de iterações variou de 377.469 a 585.926.

Tabela 1 – Resultados de 10 execuções com chutes iniciais aleatórios ($\mu = 100$, $\alpha = 8 \times 10^{-8}$)

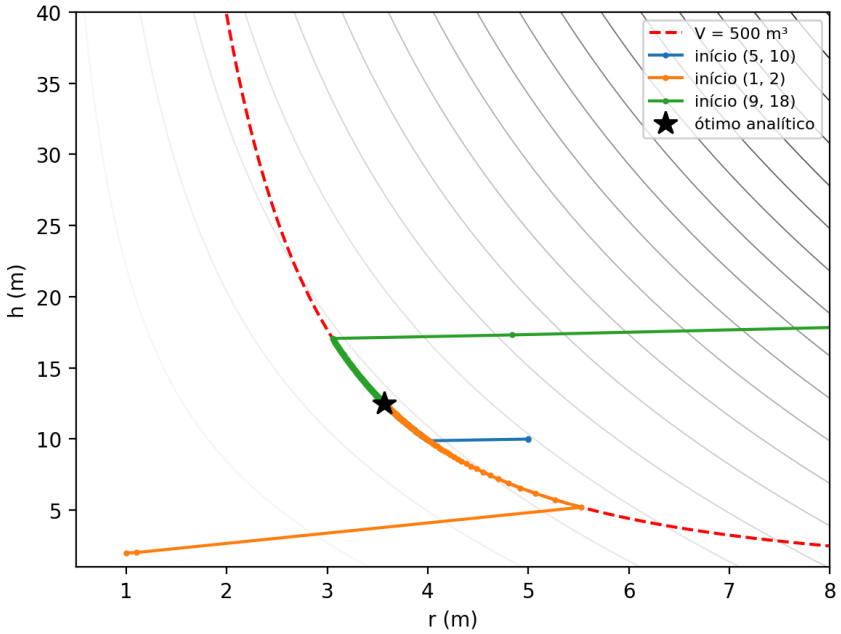
Execução	Iterações	r (m)	h (m)	V (m³)	Custo (R$)
1	584.280	3,5595	12,5545	499,719	42.009,47
2	577.585	3,5595	12,5545	499,719	42.009,47
3	377.469	3,5573	12,5698	499,719	42.009,62
4	585.926	3,5595	12,5545	499,719	42.009,47
5	582.770	3,5595	12,5545	499,719	42.009,47
6	518.368	3,5595	12,5545	499,719	42.009,47
7	577.752	3,5595	12,5545	499,719	42.009,47
8	517.162	3,5595	12,5545	499,719	42.009,47
9	411.345	3,5595	12,5545	499,719	42.009,47
10	585.435	3,5595	12,5545	499,719	42.009,47

Fonte: Elaborado pelos autores (2026).

O ponto ótimo numérico é, portanto, $r = 3{,}5595$ m e $h = 12{,}5545$ m, com custo mínimo de R$ 42.009,47. Em relação à solução analítica ($r^* = 3{,}5693$ m, $h^* = 12{,}4926$ m, $f^* =$ R$ 42.024,95), o raio difere em cerca de 0,28% e a altura em cerca de 0,5%. O custo numérico é ligeiramente menor que o analítico, e isso não indica um resultado melhor que o ótimo: o volume obtido, 499,719 m³, está 0,281 m³ abaixo do exigido. Essa pequena violação é uma característica da penalização exterior, em que a solução se aproxima da região viável pelo lado de fora. No ótimo, o multiplicador de Lagrange vale aproximadamente R$ 56,03 por m³ de volume, e a violação esperada é esse valor dividido por 2μ, ou seja, cerca de 0,280 m³, em concordância com o valor obtido.

O fato de execuções iniciadas em pontos muito diferentes, viáveis e inviáveis, chegarem ao mesmo ponto confirma que o problema não apresenta mínimos locais que prendam o algoritmo, em coerência com a convexidade da função reduzida da Equação (3).

Figura 1 – Trajetórias do método do gradiente no plano r–h ($\mu = 100$; marcadores a cada 2.000 iterações)



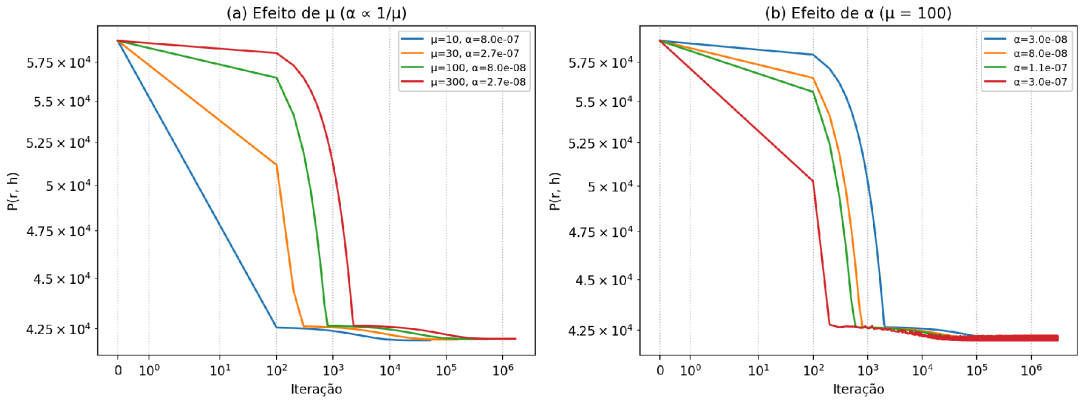
Fonte: Elaborado pelos autores (2026).

4.2 Evolução da convergência

A Figura 1 mostra que o método tem duas fases bem distintas. Na primeira, o algoritmo se aproxima rapidamente da curva de volume constante $V = 500$ m³. Quando o chute inicial é inviável, como em (1; 2), a penalidade gera gradientes grandes e o iterado é lançado até a vizinhança da curva. Na segunda fase, o iterado desliza lentamente ao longo da curva, em direção ao ponto de menor custo, por um vale estreito e alongado da função P. É nessa segunda fase que se concentra quase todo o número de iterações.

Esse comportamento explica o custo computacional elevado. Na direção do gradiente da restrição, a curvatura de P é da ordem de $2\mu[(2\pi rh)^2 + (\pi r^2)^2] \approx 1{,}6 \times 10^7$ para $\mu = 100$, enquanto ao longo da curva ela é da ordem de 10^2. A razão entre as duas curvaturas, de aproximadamente 10^5, caracteriza um problema mal condicionado, para o qual o método do gradiente com passo fixo é intrinsecamente lento: o passo é limitado pela direção mais íngreme, mas o progresso é determinado pela direção mais suave.

Figura 2 – Evolução de $P(r, h)$ com as iterações para diferentes valores de μ (a) e de α (b)



Fonte: Elaborado pelos autores (2026).

4.3 Influência da constante de penalidade μ

A Tabela 2 apresenta os resultados para μ entre 10 e 300, com $\alpha = 8 \times 10^{-6}/\mu$ e chute inicial (5, 10). Todos os casos convergiram, mas a qualidade da solução e o esforço computacional variam em sentidos opostos.

Tabela 2 – Efeito da constante de penalidade μ (chute inicial (5; 10))

μ	α	Iterações	r (m)	h (m)	V (m³)	Custo (R$)
10	$8{,}0 \times 10^{-7}$	52.507	3,5627	12,4687	497,192	41.867,45
30	$2{,}7 \times 10^{-7}$	158.180	3,5651	12,4987	499,064	41.972,52
100	$8{,}0 \times 10^{-8}$	534.230	3,5595	12,5545	499,719	42.009,47
300	$2{,}7 \times 10^{-8}$	1.663.709	3,5392	12,7036	499,906	42.022,62

Fonte: Elaborado pelos autores (2026).

Para $\mu = 10$, o volume final é de 497,192 m³, com violação de 2,81 m³ (0,56%) e custo 0,37% abaixo do valor analítico, o que torna a solução inaceitável como projeto. Ao aumentar μ, a violação cai aproximadamente com $1/\mu$: 0,93 m³ para $\mu = 30$, 0,28 m³ para $\mu = 100$ e 0,094 m³ para $\mu = 300$, e o custo se aproxima de R$ 42.024,95. Em contrapartida, como α precisa diminuir na mesma proporção, o número de iterações cresce de forma aproximadamente linear com μ, de 52.507 para 1.663.709, um aumento de cerca de 32 vezes para um aumento de 30 vezes em μ. Os valores de r e h para $\mu = 300$ ficam mais afastados do ótimo analítico do que os de $\mu = 100$. Atribui-se isso ao erro das diferenças finitas adiantadas, cujo erro de truncamento no gradiente é da ordem de $\Delta/2$ vezes a curvatura de P, que cresce com μ e pode ultrapassar a tolerância de 1,0 do critério de parada, deslocando o ponto em que o gradiente numérico se anula. Assim, aumentar μ melhora a viabilidade, mas não melhora necessariamente a precisão ao longo da curva

4.4 Influência do tamanho do passo α

A Tabela 3 mostra os resultados para $\mu = 100$ com quatro valores de α. Os três primeiros convergiram ao mesmo ponto (r = 3,5595 m, h = 12,5545 m, R$ 42.009,47), diferindo apenas no número de iterações, que é inversamente proporcional a α: 1.424.618 iterações para 3×10^{-8}, 534.230 para 8×10^{-8} e 388.530 para $1{,}1 \times 10^{-7}$.

Tabela 3 – Efeito do tamanho do passo α (μ = 100; chute inicial (5; 10))

α	Iterações	r (m)	h (m)	V (m³)	Custo (R$)	Situação
$3{,}0 \times 10^{-8}$	1.424.618	3,5595	12,5545	499,719	42.009,47	Convergiu
$8{,}0 \times 10^{-8}$	534.230	3,5595	12,5545	499,719	42.009,47	Convergiu
$1{,}1 \times 10^{-7}$	388.530	3,5595	12,5545	499,719	42.009,47	Convergiu
$3{,}0 \times 10^{-7}$	3.000.000	3,5761	12,5039	502,366	42.157,42	Não convergiu

Fonte: Elaborado pelos autores (2026).

Com $\alpha = 3 \times 10^{-7}$, o método não atingiu o critério de parada em 3.000.000 de iterações. O iterado permaneceu limitado na vizinhança do ótimo, mas terminou com volume de 502,366 m³ e custo de R$ 42.157,42, sem a estabilização esperada. Esse resultado é coerente com o limite de estabilidade do método do gradiente com passo fixo, que exige α menor que 2 dividido pela maior curvatura da função. Com $\mu = 100$, esse limite é de aproximadamente $1{,}25 \times 10^{-7}$, valor entre $1{,}1 \times 10^{-7}$, que convergiu, e 3×10^{-7}, que não convergiu. Assim, o melhor desempenho foi obtido com o maior passo ainda estável, e passos acima do limite inutilizam o método.

4.5 Desempenho computacional

O script completo, com as 10 execuções aleatórias e os oito casos dos estudos de parâmetros, levou cerca de 30 segundos em um computador comum. Considerando que cada iteração exige três avaliações de P e que há de centenas de milhares a alguns milhões de iterações por caso, o tempo é aceitável para um problema de duas variáveis, mas indica que o método não é eficiente para problemas maiores ou mais rígidos.

5 CONCLUSÃO

O algoritmo implementado resolveu o problema do tanque cilíndrico. Em todas as execuções, o método do gradiente com derivadas por diferenças finitas e função de penalização convergiu para a mesma região, com raio de aproximadamente 3,56 m, altura de aproximadamente 12,55 m e custo próximo de R$ 42.009,47. Esses valores são consistentes com a solução analítica (3,5693 m; 12,4926 m; R$ 42.024,95), e a pequena diferença é

explicada pela violação residual do volume (0,28 m³), inerente à penalização exterior com μ finito. Para fins de projeto, convém adotar dimensões levemente maiores que as numéricas, que garantam volume igual ou superior a 500 m³, como as da solução analítica.

Quanto ao desempenho, o método é robusto em relação ao chute inicial, mas lento: as execuções exigiram entre $3{,}8 \times 10^5$ e $5{,}9 \times 10^5$ iterações. A causa é o mau condicionamento da função auxiliar, que se agrava com μ. Essa lentidão é o principal custo do método do gradiente com passo fixo para esse tipo de problema.

As maiores dificuldades estiveram na sintonia dos parâmetros. A constante μ impõe um compromisso entre precisão e custo: valores pequenos produzem soluções inviáveis, e valores grandes tornam o problema mais rígido e o método mais lento. O passo α está acoplado a μ e precisa ficar abaixo de um limite de estabilidade que decresce com μ, sendo ineficiente se for muito pequeno e instável se for grande. Também foi necessário escolher a tolerância do gradiente, que pode interromper o método antes do fim do deslizamento ao longo do vale, como na execução 3, e conviver com o erro das diferenças adiantadas, que cresce com μ.

Como melhorias, sugerem-se o uso de uma sequência crescente de valores de μ, com reaproveitamento do ponto anterior, a escolha adaptativa do passo por busca linear, o emprego de diferenças centrais, o escalonamento das variáveis e a adoção de métodos que usem informação de segunda ordem ou momento, que reduzem o efeito do mau condicionamento.

APÊNDICE A – CÓDIGO-FONTE

```
import math
import random as rd
import numpy as np
import matplotlib.pyplot as plt



#Modelo do tanque
def f(r, h):
    return 150 * math.pi * r**2 + 100 * 2 * math.pi * r * h + 200 * math.pi * r**2


def g(r, h):
    return 500 - math.pi * r**2 * h


def P(r, h, mu):
    viol = max(0.0, g(r, h))
    return f(r, h) + mu * viol * viol


def gradiente(r, h, mu, delta=1e-5):
    """Derivadas parciais por diferenças finitas adiantadas (eqs. 4 e 5)."""
    p0 = P(r, h, mu)
    return (P(r + delta, h, mu) - p0) / delta, (P(r, h + delta, mu) - p0) / delta



#Método do gradiente (eqs. 6 e 7): mu e alpha fixos


def metodo_gradiente(mu, alpha, r0, h0, tol=1.0, max_iter=3000000):
    r, h = r0, h0
    hist = []
    for k in range(max_iter):
        if k % 100 == 0:
            hist.append(P(r, h, mu))
        dr, dh = gradiente(r, h, mu)
        if math.hypot(dr, dh) < tol:
            return r, h, k, hist, True
        r, h = r - alpha * dr, h - alpha * dh
        if not (math.isfinite(r) and math.isfinite(h)) or abs(r) > 1e6 or abs(h) > 1e6:
            return r, h, k, hist, False
    return r, h, max_iter, hist, False


def resumo(mu, alpha, r0, h0):
    r, h, it, hist, ok = metodo_gradiente(mu, alpha, r0, h0)
    return dict(mu=mu, alpha=alpha, r=r, h=h, it=it, hist=hist, ok=ok,
                custo=f(r, h), V=math.pi * r**2 * h)



#Experimentos


def multiplas_execucoes(n, mu, alpha, tol_viab=0.5):
    res = [resumo(mu, alpha, rd.uniform(0.1, 10), rd.uniform(0.1, 20)) for _ in range(n)]
    for i, s in enumerate(res, 1):
        viavel = s["ok"] and 500 - s["V"] <= tol_viab
        s["viavel"] = viavel
        print(f"Exec {i:02d} | it: {s['it']:6d} | r: {s['r']:.4f} | h: {s['h']:.4f} | "
              f"V: {s['V']:.3f} | Custo: R$ {s['custo']:,.2f} | {'viável' if viavel else 'INVIÁVEL/não
conv.'}")
    ok = [s for s in res if s["viavel"]]
    if not ok:
        print("Nenhuma execução viável."); return
    c = np.array([s["custo"] for s in ok])
    best = min(ok, key=lambda s: s["custo"])
    print(f"\nViáveis: {len(ok)}/{n} | Custo mín: R$ {c.min():,.2f} | médio: R$ {c.mean():,.2f} | "
          f"máx: R$ {c.max():,.2f}")
    print(f"Ótimo: r = {best['r']:.4f} m, h = {best['h']:.4f} m, V = {best['V']:.3f} m³")


def estudo(casos, r0, h0, titulo, ax, rotulo):
    print(f"\n{titulo}\n{'mu':>8} {'alpha':>9} {'iter':>7} {'r':>8} {'h':>8} {'V':>9} {'custo':>12}  status")
    for mu, alpha in casos:
        s = resumo(mu, alpha, r0, h0)
        print(f"{mu:8.0e} {alpha:9.1e} {s['it']:7d} {s['r']:8.4f} {s['h']:8.4f} {s['V']:9.3f} "
              f"{s['custo']:12,.2f}  {'convergiu' if s['ok'] else 'NÃO convergiu/divergiu'}")
        if all(math.isfinite(v) for v in s["hist"]):
            ax.plot(np.arange(len(s["hist"])) * 100, s["hist"], label=rotulo(mu, alpha))
    ax.set(xscale="symlog", yscale="log", xlabel="Iteração", ylabel="P(r, h)", title=titulo)
    ax.grid(True, ls=":"); ax.legend()


def main():
    rd.seed(42)  # semente fixa para reprodutibilidade
    try:
        n = int(input("Digite o número de execuções: "))
    except ValueError:
        n = 10; print("Entrada inválida. Usando 10 execuções.")
    MU, ALPHA = 100.0, 8e-8
```

```
    print("=" * 70 + "\n MÚLTIPLAS EXECUÇÕES (chutes iniciais aleatórios) \n" + "=" * 70)
    multiplas_execucoes(n, MU, ALPHA)


    fig, axs = plt.subplots(1, 2, figsize=(14, 5))
    estudo([(m, 8e-6 / m) for m in (10, 30, 100, 300)], 5.0, 10.0,
           "Efeito de mu (alpha ~ 1/mu)", axs[0], lambda m, a: f"mu={m:.0e}, alpha={a:.1e}")
    estudo([(MU, a) for a in (3e-8, 8e-8, 1.1e-7, 3e-7)], 5.0, 10.0,
           f"Efeito de alpha (mu={MU:.0f})", axs[1], lambda m, a: f"alpha={a:.1e}")
    plt.tight_layout(); plt.show()


if __name__ == "__main__":
    main()
```